Unidad 4 / Escenario 7

Lectura fundamental

Tipos de datos compuestos definidos por el usuario

Contenido

1 Creación de clases en Java 1

2 Métodos 2

3 Agregación de datos 6

4 Problemas propuestos 8

Palabras claves: datos, tipos de datos abstractos, métodos y datos compuestos.

En este documento se hace una introducción a las herramientas básicas para la definición, implementación y uso de tipos de datos personalizados. Estas herramientas son muy importantes, puesto que en casi todo modelamiento en la práctica de un programador, cuando se analiza un problema o sistema del mundo real, resulta imposible o muy difícil de manejar la información solo con tipos de dato nativos y clases definidas de manera estándar en el lenguaje de programación. Esta necesidad está presente de manera recurrente en el modelamiento e implementación en gran parte de las soluciones informáticas y algoritmos. Los tipos de datos definidos por el usuario tienen un origen en herramientas como los *struct* de C (cppreference, 2015). Estas herramientas permiten acceder a un conjunto de valores a través de una misma variable, que pueden ser, por ejemplo, las características de una persona, como nombres, apellidos, fecha de nacimiento, tipo de documento, documento, etc.

Las aproximaciones de creación de tipos de datos definidos por el programador evolucionaron en lo que se conoce como el paradigma de programación orientado por objetos.

1. Creación de clases en Java

Como primer ejemplo, considere una versión simplificada de lo que es un tipo de dato "Persona". Como se puede ver en el código 1, en la misma jerarquía que la *main* en la clase *ClasesFirstExample*, está una construcción que inicia con las palabras ***private static class Person***. Esta es una de varias formas en Java de declarar un nuevo tipo de dato; la palabra reservada *private* es un modificador de privacidad y sirve para que no se pueda acceder a ese dato fuera de la clase *ClaseFirstExample*. El modificador *static* se usa aquí para poder acceder a la declaración e instanciación de ejemplares de la clase *Person* sin necesidad de instanciar la clase *ClasesFirstExample*. Finalmente, en esta línea está la palabra reservada *class* seguida del nombre escogido para el tipo de dato *Person*. Es importante que los nombres de clases o tipos de datos estén en singular y, aunque no es obligatorio para su correcta compilación, sus nombres deben obedecer el estándar *UpperCammelCase*, al igual que el resto de clases estándar de Java. Esto último puede cambiar de un lenguaje de programación a otro, teniendo en cuenta las buenas prácticas de programación.

Aunque en el ejemplo 1 se muestra la implementación de la clase dentro del mismo archivo que está el método *Main*, la recomendación general, referente a las buenas prácticas de diseño/programación/desarrollo, es que cada clase diseñada vaya en un archivo aparte. El archivo de cada clase debe llamarse igual que la clase pública. Para el ejemplo mostrado, equivaldría a tener dos archivos, el primero con el *Main* y el segundo con la clase, como se muestra en los códigos 3 y 2, respectivamente.

1.1. Atributos

Dentro de la clase *Person*, se pueden ver declaradas cuatro variables, de las cuales *peopleCount* es pública y estática, mientras que las demás son privadas y no estáticas (no tienen el modificador *static*). Estas variables se conocen como atributos o campos de una clase. En el proceso de abstracción y diseño de clases se deben identificar las características de los ejemplares de lo que será el tipo de datos a implementar. En este caso, para simplificar el ejemplo, se escogieron solo tres atributos: el *id* de una persona (*id*), su nombre (*name*) y apellido (*lastname*). El campo estático **peopleCount** se usa para llevar una cuenta de cuántas personas se han instanciado, es decir, cuántos ejemplares de la clase *Person* se han creado. Aunque los atributos de una persona pueden ser declarados públicos o se podría incluso omitir el modificador *private*, por buenas prácticas de programación se recomienda que sean privados, para que su modificación se haga únicamente dentro de la clase. Esta práctica tiene que ver con el encapsulamiento como uno de los pilares fundamentales de programación orientada por objetos. Una clase puede tener muchos atributos y de tantos y diferentes tipos como se necesite en el proceso de abstracción y diseño de clases en Java.

2. Métodos

Dentro de una clase se pueden crear métodos con diferentes propósitos. Estos métodos también se conocen en el ámbito de la programación orientada por objetos como comportamientos. Dentro de estos, los más comunes en el diseño de clases son los métodos constructores, los *getters*, los *setters* y un método que retorne una representación como cadena de la instancia de la clase; es decir, el método *toString*. En esta sección se explicará de manera breve cada uno de estos tipos de métodos, sus usos y recomendaciones de implementación.

2.1. Métodos constructor

Los constructores son métodos que sirven para crear una instancia (un ejemplar) de una clase. En Java y en muchos lenguajes de programación tienen el mismo nombre de la clase. Pueden recibir un conjunto de parámetros o no hacerlo y pueden ser uno o más constructores por clase. En el caso del ejemplo mostrado en el Código 1, se ve un constructor que recibe tres argumentos (*id*, *name*, *lastName*). Los argumentos que recibe un constructor se usan generalmente para darle valores iniciales a los atributos de la instancia que está siendo creada. En el caso del ejemplo, se usa la palabra reservada *this* para acceder a las variables miembro (atributos) de la clase Persona; es por esto que las instrucciones como **this.id = id**; se deben entender como una asignación del argumento que llega (*id*) al atributo de la instancia (*this.id*). En el constructor del ejemplo se puede ver también la instrucción **Person.peopleCount++**; esta instrucción es diferente a las otras tres en cuanto a que accede al atributo **peopleCount** a través del nombre de la clase. Este tipo de acceso a atributos solo se puede hacer cuando el atributo es estático, puesto que habrá solo uno sin importar cuantas instancias haya; en nuestro ejemplo, sin importar cuantas personas se creen.

Como ejercicio se sugiere eliminar las palabras reservadas **this.** del constructor y analizar el resultado del programa. Aunque el programa compila, es probable que el ambiente integrado de desarrollo usado muestre algunas advertencias. Otra práctica que vale la pena hacer es cambiar la forma en que se accede al campo **peopleCount** de **Person.peopleCount** a **this.peopleCount** y analizar el resultado y el porqué de la advertencia que probablemente arroje el ambiente integrado de desarrollo. Vale la pena anotar que, debido a que el constructor está dentro de la clase, en este caso se puede acceder a ese campo solo con el nombre de la variable, es decir, se podría tener solo **peopleCount++**; en el constructor y el programa funcionaría normalmente. Esta última modificación no es cierta para el main en donde sí es obligatorio el uso del nombre de la clase (*Person*) para acceder al campo **peopleCount**, esto porque el *main* está fuera de la clase.

2.2. *Métodos getter y setter*

Después de los constructores, los métodos más comunes en diseño de clases y tipos de datos personalizados son, tal vez, los *getters* y *setters*. Los métodos conocidos como *getters* sirven para acceder a los atributos de una instancia (o a alguna representación de ellos). En el ejemplo de Código 1, la clase tiene los *getters, getName* y *getLastName*. Aunque el nombre a escoger puede ser diferente al mostrado, como buena práctica de programación a obedecer, estos nombres deben ir siempre escritos en el estándar **lowerCammelCase** y con la palabra *get* al principio, seguida del nombre del atributo en el estándar **UpperCammelCase**.

Así como los métodos *getter* sirven para obtener/recuperar un atributo de una instancia de una clase, los métodos *setter* se usan y se crean como una manera de establecer/actualizar el valor de un atributo en una instancia. En el caso del ejemplo 1 se puede ver el método *setName*, que sirve para actualizar/cambiar el nombre de una persona. De manera similar al nombramiento de los *getters*, los *setters* pueden ser nombrados con cualquier identificador válido, pero se recomienda, por buenas prácticas de programación, que sean nombrados iniciando con *set* y seguidos por el atributo que actualizan en estándar **UpperCammelCase**.

Por último, en el diseño de una clase no necesariamente todos los atributos deben tener getter y setter; esto depende del problema que se esté resolviendo y de cómo se vayan a usar los atributos. En general, se recomienda, como buena práctica de programación, crear estos métodos siguiendo el principio del mínimo privilegio, el cual establece, en este caso, que se debe escribir un *getter* para un atributo si y solo si se necesita su valor fuera de la clase y, de manera similar, se debe escribir un *setter* para un atributo si y solo si hay seguridad de que se necesitará una actualización después de la instanciación de una clase dada.

2.3. Método *toString*

Otro método de uso común, en especial para propósitos de depuración de programas, es el *toString*. Este método, escrito con la firma public **String toString()** sobreescribe el método de la clase Object. Esto quiere decir, para propósitos de nuestro ejemplo, que cada vez que se imprima una instancia de la clase *Person*, se mostrará la información que retorna este método. Como ejercicio, se recomienda eliminar el método *toString* del ejemplo de código y analizar el resultado de la ejecución del programa.

```java
public class ClasesFirstExample{

    private static class Person{
        public static int peopleCount = 0;
        private long id;
        private String name;
        private String lastName;
        public Person(long id, String name, String lastName){
            Person.peopleCount++;
            this.id = id;
            this.name = name;
            this.lastName = lastName;
        }
        @Override
        public String toString() {
            return id+" "+this.name+" "+this.lastName;
        }
        public String getName() {
            return name;
        }
        public String getLastName() {
            return lastName;
        }
        public void setName(String newName) {
            this.name = newName;
        }
    }
    public static void main( String args[] ){

        Person ralph, eli;
        System.out.println("There is now "+Person.peopleCount+" people.");
        ralph = new Person(1000,"Ralph","Jones");
        System.out.println("There is now "+Person.peopleCount+" people.");
        eli = new Person(2000,"Eli","Smith");
        System.out.println("There is now "+Person.peopleCount+" people.");
        System.out.println(ralph);
        System.out.println(eli);
        System.out.println("The lastname of "+
                eli.getName()+" is "+eli.getLastName());
        eli.setName("Elisabeth");
        System.out.println(eli);

    }
}
```

Código 1: Ejemplo de uso de Clases

```java
public class Person{
    public static int peopleCount = 0;
    private long id;
    private String name;
    private String lastName;
    public Person(long id, String name, String lastName){
        Person.peopleCount++;
        this.id = id;
        this.name = name;
        this.lastName = lastName;
    }
    @Override
    public String toString() {
        return id+" "+this.name+" "+this.lastName;
    }
    public String getName() {
        return name;
    }
    public String getLastName() {
        return lastName;
    }
    public void setName(String newName) {
        this.name = newName;
    }
}
```

Código 2: Ejemplo de uso de Clases (*Clase Person*). Archivo Person.java

```java
public class MainClass{
    public static void main( String args[] ){
        Person ralph, eli;
        System.out.println("There is now "+Person.peopleCount+" people.");
        ralph = new Person(1000,"Ralph","Jones");
        System.out.println("There is now "+Person.peopleCount+" people.");
        eli = new Person(2000,"Eli","Smith");
        System.out.println("There is now "+Person.peopleCount+" people.");
        System.out.println(ralph);
        System.out.println(eli);
        System.out.println("The lastname of "+
                eli.getName()+" is "+eli.getLastName());
        eli.setName("Elisabeth");
        System.out.println(eli);
    }
}
```

Código 3: Ejemplo de uso de Clases (*Clase Main*). Archivo MainClass.java

3. Agregación de datos

Como extensión de la idea de tipo de dato compuesto, se puede considerar un tipo un poco más complejo. Se podría pensar entonces en un tipo de dato que sea la abstracción de un conjunto o colección de otros tipos de datos también compuestos, por ejemplo: de manera muy simplificada, lo que sería el tipo de dato familia, que, en su versión más sencilla, es un conjunto de personas.

Al tomar como punto de partida el ejemplo anterior de la clase persona, adicionar el *getter* a la clase persona para el atributo *id* y crear la clase familia, la cual es una agregación de personas, se puede ver un ejemplo de definición de tipo de dato personalizado a partir de otro tipo de dato también personalizado. En general, es bastante común usar contenedores para almacenar instancias de tipos de datos personalizados y a través de esto hacer agregaciones y composiciones de datos.

En el programa mostrado en el Código 4 se puede ver cómo se diseña la clase *Family*, la cual, en su instanciación, crea un ArrayList de personas que inicialmente está vacía. La clase tiene un constructor sin parámetros cuya única tarea es incializar el ArrayList donde se almacenarán las personas. La clase también tiene un método para agregar personas a la familia, un método para consultar una persona por *ID*, que en caso de no encontrar el *id* retorna *null*, y también tiene sobrescrito el método *toString*, el cual retorna una representación fácil de leer e interpretar de la instancia que lo use. Este método *toString* es el invocado cuando se "imprime" a la salida estándar la instancia creada de *Family* en el método *main*. Finalmente, en este método se crea una referencia a *Family*, se instancia y se le agregan cuatro elementos tipo *Person*. Luego, se muestra la instancia y se hacen dos consultas, una para una persona que está efectivamente en la familia (id=3) y otra para alguien que no esté (id=6). Se recomienda, como siempre, hacer un seguimiento paso a paso del programa y ejecutarlo y compilarlo con ligeras modificaciones al código para asegurar la apropiación de los conceptos dados.

```java
import java.util.ArrayList;
public class Family {
    private ArrayList<Person> people;
    public Family(){
        this.people = new ArrayList<Person>();
    }
    public void addPersonToFamily( Person p ){
        this.people.add(p);
    }
    public Person getPerson( long id ){
        for( Person p : this.people )
            if( p.getID() == id )
                return p;
        return null;
    }
    public int getFamilySize(){
        return this.people.size();
    }
    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder();
        sb.append("This Family has "+this.people.size()+" mebers:\n");
        for (int i = 0; i < people.size(); i++)
            sb.append("Member No "+(i+1)+" : "+people.get(i)+"\n");
        return sb.toString();
    }
    public static void main(String[] args) {
        Family myFamily = new Family();
        Person a = new Person(1,"John","Smith");
        Person b = new Person(2,"Mary","Smith");
        Person c = new Person(3,"Charles","Smith");
        Person d = new Person(4,"Johana","Smith");
        myFamily.addPersonToFamily(a);
        myFamily.addPersonToFamily(b);
        myFamily.addPersonToFamily(c);
        myFamily.addPersonToFamily(d);
        System.out.println(myFamily);
        System.out.println(myFamily.getPerson(3));
        System.out.println(myFamily.getPerson(6));
    }
}
```

Código 4: Ejemplo de uso de Clases agregación de clases.

4. Problemas propuestos

Casi todos problemas de esta sección constan de un enunciado en el que se solicita el diseño e implementación de un programa en Java, que tome como entrada ciertos datos leídos desde su entrada estándar y produzca como única salida los datos solicitados, que pueden ser cálculos o transformaciones hechas con los datos de entrada. En todos los problemas propuestos se debe asumir que las entradas cumplirán con lo estipulado en la sección "`Descripción de entrada`" de cada problema. Dicho esto, se debe entonces diseñar e implementar un programa que cumpla con lo estipulado en la sección "`Descripción de salida`", que sea únicamente lo solicitado allí, nada más.

Teniendo en cuenta que en este Escenario se abordan tipos de datos definidos por el programador, se espera que en todas las soluciones se usen de alguna manera. Como requisito adicional, toda la lectura de datos (uso de *Scanner*) y toda impresión de datos a la salida estándar (uso de *System.out*) debe hacerse en el método *main*; es decir, en los demás métodos no puede haber ni lectura ni impresión de datos.

1. Centro de masas. Dados un conjunto de puntos con masa en el plano, hacer un programa que lea estos datos y calcule y muestre el centro de masas de estos puntos. En este problema se debe crear la clase *Point* y la clase *PointSet*. Vale la pena aclarar que, de manera estándar, Java tiene una clase *Punto* que se podría usar aquí (Oracle, 1999). El centro de masas está dado por la siguiente fórmula:

$$x_{cm} = \frac{\sum_{i=1}^{n} m_i * x_i}{\sum_{i=1}^{n} m_i}$$
$$y_{cm} = \frac{\sum_{i=1}^{n} m_i * y_i}{\sum_{i=1}^{n} m_i}$$

- Descripción de entrada: un número entero **n** seguido de **n** triplas de números. Cada tripla es un punto con su masa, primero la coordenada **x**, luego la coordenada **y** y después la masa **m** para ese punto.
- Descripción de salida: dos números, la coordenada **x** del centro de masa y la coordenada y del centro de masa de los puntos leídos.
- Ejemplo de entrada:
  5
  10 10 2
  10 20 4
  10 30 8
  10 40 1
  10 50 16
- Ejemplo de salida:
  10 38.065

2. Diseño de clase triángulo. Diseñar una clase que sea la abstracción de un triángulo en el plano cartesiano. Aquí, se espera que los atributos de la clase triángulo sean tres puntos en el plano cartesiano. Adicional a los atributos, la clase debe tener comportamientos como: obtener lados, obtener área, obtener perímetro, obtener punto del centro de masas. Se recomienda usar la clase punto creada en el problema anterior. Diseñar, también, un método *main* y verificar el funcionamiento de la clase con la instanciación de varios ejemplares de la clase triángulo.

3. Diseño de la clase círculo. Diseñar una clase que sea la abstracción de un círculo en el plano cartesiano. Esta clase debe tener dos atributos, un punto que será su centro, y un real que será su radio. Se recomienda usar la clase punto creada para problemas anteriores.

4. Espectro. Diseñar e implementar un programa que tome como entrada 5 números enteros positivos y produzca como salida un patrón como el que se muestra en el ejemplo. En este problema se requiere que se diseñe una clase que sea una abstracción de un Espectro. Esta clase debe tener como atributos los cinco valores dados por la entrada. Como ejercicio adicional, se puede considerar la generalización del problema a un número arbitrario de columnas (no necesariamente 5).

- Descripción de entrada: un número **n** entero positivo, seguido de **n** números enteros no negativos. Todos entre 0 y 100.
- Descripción de salida: el patrón según se muestra en el ejemplo. El primer número leído **n** será la cantidad de columnas de la matriz a imprimir. Los siguientes **n** números serán la cantidad de numerales que salen en cada columna. Así, en el ejemplo, en la primera columna hay 3 numerales, en la segunda un numeral, en la tercera 8 numerales (el mayor valor da la cantidad de filas), la penúltima columna muestra 4 numerales y la última muestra 2 numerales.
- Ejemplo de entrada:
  5
  3 1 8 4 2

```
..#..
..#..
..#..
..#..
..#..
..##.
#.##.
#.###
#####
```

Código 5: Patrón a imprimir en el problema

5. Se desea hacer un programa que tome como entrada los datos de estudiantes (nombres completos, números de documento) y una serie de notas y produzca una lista Estudiante-Promedio, ordenada de mejor a peor promedio de notas, y muestre el resultado a manera de tabla con tres columnas: nombre, cédula y promedio. El formato del archivo de entrada es el siguiente:

- En la primera línea del archivo de entrada habrá un número entero positivo **n**; esta será la cantidad de líneas que siguen. Cada línea representará la información de un estudiante y tendrá el siguiente formato: nombre completo - número de documento - serie de notas entre 0 y 5, separadas por espacios.
- Descripción de salida: la lista según lo solicitado.
- Ejemplo de entrada:
  4
  Rafael Alberto Velasquez-889993893-4 3 4 4.5 4 2 1
  Jose Figueroa-2334234-4 3 1 3 5
  Diego Castilla-232234-3 4 1 2 3 3 2 1 2.3 2.3
  Alberto Baron-234213-3 3 1
- Ejemplo de salida:
  Alberto Baron 234213 4.33333
  Rafael Alberto Velasquez 889993893 3.21429
  Jose Figueroa 2334234 3.2
  Diego Castilla 232234 2.36

Referencias

Fernández, H. (2012). *Programación orientada a objetos usando Java*. Ecoe Ediciones.

Goytia, J. (2014). *Programación orientada a objetos C++ y Java: un acercamiento interdisciplinario*. Grupo Editorial Patria.

Llinás, L. (2010). Todo lo básico que debería saber: sobre programación orientada a objetos en java. Ediciones de la U.

Oracle, C. (1999). *Point in java Documentation*. Recuperado el 30 de abril de 2018, desde https://docs.oracle.com/javase/7/docs/api/java/awt/Point.html

Reference, C. (2015). *Struct declaration*. Recuperado el 30 de abril de 2018, desde https://en.cppreference.com/w/c/language/struct

Regino, E. O. (2015). *Lógica de programación orientada a objetos*. Ecoe Ediciones.

Serrano, J., Abril, A. & Bellas, P. (2014). *Diseñar y programar, todo es empezar: una introducción a la Programación Orientada a Objetos usando UML y Java*. Grupo Editorial Patria.

Valbuena, S., Torres, S., & Rodríguez, L. (2010). *Programación orientada a objetos.* Ediciones

INFORMACIÓN TÉCNICA

Ingeniería y Ciencias Básicas

Módulo: Conceptos Fundamentales de Programación
Unidad 4: Tipos y contenedores de datos
Escenario 7: Tipos de datos compuestos definidos por el usuario

Autor: Rafael Niquefa

Asesor Pedagógico: Angie Laitón
Diseñador Gráfico: Rafael Niquefa
Asistente: Alejandra Morales

Este material pertenece al Politécnico Grancolombiano. Por ende, es de uso exclusivo de las Instituciones adscritas a la Red Ilumno. Prohibida su reproducción total o parcial.